2025年度　卒 業 論 文

逆強化学習を用いた
継続的な貢献を行うLTCの予測モデル

2026年2月10日

システム工学科
(学生番号: 60276189)

橋本 一輝

和歌山大学システム工学部

概 要

オープンソースソフトウェア (OSS: Open Source Software) 開発において，長期貢献者 (LTC: Long-Term Contributor) はプロジェクトの継続的な発展と持続可能性を高める上で重要な役割を担っている．しかし，多くの開発者はプロジェクトに参加後，早期に離脱してしまうことが多いため，LTC となり得る開発者を早期に特定し適切なサポートをする必要がある．従来研究では，予測時点から 1 年以降や 3 年以降に貢献があるか否かを予測する．しかし，1 年後以降や 3 年後以降のように単一時点以降に貢献があるか否かを予測するため，開発者がどの期間で貢献するかが明確でなく，特定の期間において開発者が貢献するか否かを予測することができない．そこで，本研究では，開発者の貢献をより詳細に捉えるために，評価期間を区切り，将来の特定の期間内において，継続的な貢献があるか否かを連続的に予測する．具体的には，コードレビューにおける貢献に着目し，開発者がコードレビュー依頼を継続的に承諾するか否かを開発者の意思決定プロセスと捉え，逆強化学習 (IRL) を用いて報酬関数を推定することにより予測モデルを構築する．

ケーススタディとして，OpenStack/Nova プロジェクトを対象とし，報酬関数に基づき予測を行い，従来手法で用いられたランダムフォレスト (RF) と比較した結果，最大値では劣るものの，予測する期間を変更した多くのパターンで RF の精度を上回った．また，提案手法において評価期間が近い場合に関しては貢献量などの量的な特徴量が重視され，評価期間が遠い場合に関しては協力度や応答速度などの活動量以外の特徴量が重視されることが示唆された．

目 次

第 1 章　はじめに **1**

第 2 章　持続可能な OSS 開発 **3**

2.1 OSS 開発における LTC 3

2.2 持続可能な OSS 開発に関する関連研究 3

2.2.1 OSS へ継続的に貢献する開発者の特徴 3

2.2.2 LTC 候補者の予測 4

2.2.3 開発者の離職予測に関する研究 4

2.3 本研究の位置付け 4

第 3 章　継続的な LTC 予測に向けた強化学習と逆強化学習 **6**

3.1 強化学習 6

3.2 逆強化学習 7

3.3 本研究における適用方法 7

第 4 章　LTC 予測モデルの構築 **9**

4.1 データ収集・前処理 9

4.2 学習指標の作成 9

4.2.1 基準点の設定 9

4.2.2 レビュー承諾の有無を判定 10

4.2.3 学習指標の役割 11

4.3 状態・行動特徴量の生成 11

4.4 モデル構造と学習フロー 11

4.4.1 報酬関数 12

4.4.2 損失関数による報酬関数の最適化 13

4.5 予測・評価 14

4.5.1 評価指標 14

4.5.2 クロス評価 15

第 5 章　ケーススタディ **16**

5.1 データセット 16

5.2 RQ1: 逆強化学習に基づく提案モデルは，コードレビューにおけるレビュー承諾をどの程度予測できるか 16

5.3 RQ2: 学習方法や評価期間の長さに応じて，予測モデルの精度が異なるのか 17

5.4 RQ3: 推定された報酬関数において，LTC の継続的なレビュー承諾に寄与する特徴量は何か 20

第 6 章　考察 **22**

6.1 時系列データの考慮が予測に与える影響 22

6.2 短期予測と長期予測における特徴量の違い 22

6.3 IRL/RF において予測不一致の開発者について 24

第 7 章 妥当性の脅威 **27**

7.1 内的妥当性の脅威 27

7.2 外的妥当性の脅威 27

第 8 章 おわりに **28**

第1章　はじめに

オープンソースソフトウェア (OSS: Open Source Software) は，地理的に分散した開発者によって実装，保守されている．しかし，多くの開発者は，OSS 開発組織から金銭的な利益を得ることなく，個人の知的好奇心や開発技術の学習などを動機として貢献している [1]. Raymond らは，OSS 開発の成功例として Linux を対象として調査し，不特定多数の開発者が自発的に参加する開発形態をバザール方式と名付けた [2]．この開発方式は，多様な専門知識を有する開発者の参加を促進する一方で，開発者の流動性が極めて高い [3]．したがって，開発者の動機が多様なため，時間的制約や興味の変化によって数回の貢献後に活動を停止する開発者が多い [4]．こうした開発者の離脱は，OSS プロジェクトの失敗の一要因になっている [5].

OSS を長期的に運用，保守しているプロジェクトでは，長期的に OSS プロジェクトに貢献する長期貢献者 (LTC) が重要な役割を担っている [6][7]．LTC がソフトウェアの大部分を実装しているプロジェクトも存在する [8]．また，LTC は実装に限らず，コードレビュー，新規参加した開発者のメンターの役割を担っている．一部の LTC には，リポジトリの主要なブランチにコミットする権限が与えられプロジェクトの継続的な運用を支えている [8]．このような経験が豊富な LTC は，流動性の高い組織構造を有する OSS プロジェクトでは，開発者の離脱が頻繁に発生するため，少数しか存在しないことが多い．その結果，LTC に作業負担がかかることが少なくない [9]．OSS プロジェクトの長期的な発展や，LTC への負担軽減のためにも，継続的に貢献する開発者の参加，特定が喫緊の課題である．

従来研究では，将来的に長期間にわたってプロジェクトに貢献する LTC を早期に特定する手法が提案されている [6][8]．しかし，従来研究では，開発者がプロジェクトに参加してから一定期間が経過した時点を基準とし，基準点以降にその開発者の貢献があるか否かを予測する研究が多い．そのため，開発者がどの期間でタスクに取り組み，どの期間でタスクに取り組まないのかといった詳細な貢献を予測することができず，適切なタスクを分配することが難しい．結果として，LTC への負担軽減につながりにくい．

本研究は，開発者がタスクの作業依頼を承諾するか否かを明確に判断可能なコードレビューのレビュータスクを対象に，将来のコードレビュー依頼に対してレビュアーが承諾するか否かを，開発者の継続的な意思決定プロセスと捉えた LTC 予測モデルを提案する．

本研究では，LTC 予測モデルの構築に向けて，次の 3 つの Research Questions(RQs) に回答する．

- RQ1: 逆強化学習に基づく提案モデルは，コードレビューにおけるレビュー承諾をどの程度予測できるか
- RQ2: 学習方法や評価期間の長さに応じて，予測モデルの精度が異なるのか
- RQ3: 推定された報酬関数において，LTC の継続的なレビュー承諾に寄与する特徴量は何か

RQ1 は，LTC 予測モデルの構築に逆強化学習を用いた際に，どの程度の精度で予測できるのかを調査する．RQ2 は，学習方法の変更やレビュー依頼を承諾するか否かを予測する期間を変更することによるモデルの精度差があるのか，開発者単位で評価期間ごとにレビュー承諾/非承諾を予測できるのかを調査する．RQ3 においては，予測モデルにおける特徴量の重要度を分析することにより，各特徴量がレビュアーの継続的なレビュー承諾に与える影響の重要度を調査する．

続く 2 章では，本研究の関連研究を紹介し，本研究の位置付けを明確にし，3 章では，本研究で用いる逆強化学習とそれに関連する強化学習を紹介する．4 章では，本研究の提案手法を述べ，5 章でケーススタディの結果を述べ RQ に回答する．6 章で本研究の結果を考察し，7 章で妥当性の脅威について議論し，8 章で本研究をまとめる．

第2章　持続可能なOSS開発

2.1　OSS開発におけるLTC

OSSは，ソースコードを公開し，ライセンスの条件下において，誰でも自由に閲覧・修正・再配布が可能なソフトウェアであり，世界中に地理的に分散した多くのボランティア開発者の貢献により開発が行われている [1]. 一般的な，OSS開発のフローは次のように進行する．

- ユーザや開発者が，発見したバグや機能追加などの要求をイシューやチケットとして報告
- 開発者が要求の中から対応するものを選択し，ソースコードを修正・追加し要求に対応
- 開発者が変更内容をパッチやプルリクエストとしてプロジェクトに提出

提出された変更は，プロジェクトメンバーによるコードレビューを受ける．膨大な変更提案を効率的に処理するために，個々の変更提案に対して適切なレビュアーに依頼を行う．具体的には，変更されたファイルの管理者（コードオーナー）がシステムによって自動的に依頼されたり，変更提案者やプロジェクトの管理者が適切なレビュアーに依頼したりすることにより，レビュータスクが振り分けられる．レビュアーとの議論を経て，最終的に変更が承認された場合には，変更がリポジトリに統合される．

多くの開発者がボランティアとして貢献するOSS開発プロセスにおいて，レビュー依頼を行いタスク割り当てを行なったとしても，強制力がないために依頼に対して，レビュアーがレビュー依頼を拒否したり，反応がなく放置されたりするケースが存在する．さらに，開発者は個人の事情や興味の変化などによりいつでもプロジェクトから離脱することができ，離脱や貢献といった動機は明確でないため，労働力が安定しないことや労働力を見積もることが困難であるといった課題がある．

こうした労働力が不安定な環境の中で，OSSの持続可能な開発には，プロジェクトに長期的に貢献する開発者であるLTCが重要である．しかし，OSSプロジェクトにおいてLTCとなり得る開発者は少数であり，特定の貢献者に負担が集中し，LTCの疲弊や離脱を招くリスクがある．そのため，多様な動機を持つ開発者の中から，継続的に貢献するLTCになり得る開発者を特定し，適切なサポートを行うことが重要となる [9][10].

2.2　持続可能なOSS開発に関する関連研究

2.2.1　OSSへ継続的に貢献する開発者の特徴

従来研究では，開発者が特定のプロジェクトで活動を継続する動機を調査している．Wuらによる定性的分析では，OSS開発者は利他的な動機（例：貢献による名誉や実績）や人的資本の向上に

関連する経済的動機（例：専門知識を有する開発者との協調）をもってOSSプロジェクトへの参加を継続していることを明らかにしている[11]．その他，多くの研究者がOSSプロジェクトにおける新規参加者の定着に影響を与える要因を調査している[8][12][13][14]．Birdらは，開発者がOSSプロジェクトに貢献する期間に影響を与える要因を理解するためのハザード率モデル（hazard-rate model）を提案している[13]. ケーススタディとして，Apache，Python，PostgreSQLプロジェクトでは，特定のプロジェクトで約1年間活動した開発者をリポジトリの主要なブランチにコミットする権限を与えていることを示している．特に，ApacheおよびPythonプロジェクトでは，継続的にパッチを作成している開発者に権限を与えていることを明らかにしている．このように，OSSプロジェクトは，LTC候補者に継続的な貢献を期待しており，継続的に貢献する可能性の高い開発者を早期に特定することが，プロジェクトの持続的な発展にとって不可欠であることを示唆する．

2.2.2 LTC候補者の予測

従来研究では，LTC候補者を予測する手法が提案されている[6][8][15]．Zhouらは，OSSプロジェクトに新規に参加した開発者の中からLTC候補者を特定するモデルを開発している[8]．当該研究において，LTC候補者に分類される開発者は，技術的能力，意欲，および環境に依存することを明らかにしている．特に，LTC候補者を特定するためには，プロジェクト参加後の1ヶ月間の貢献を観察し,「参加後１ヶ月間の活動量」のメトリクスを用いることで，約0.80のAUCスコアでLTCを特定することを実現している．

Eluriらは，OSSプロジェクトに新規に貢献した開発者の特徴量，および貢献したリポジトリの特徴量に基づき，開発者が3年後に貢献しているか否かを予測するモデルを開発している[6]．提案モデルは，従来手法の予測精度に比べて非常に高く，AUCが0.913という結果を得ている．当該研究の比較対象とされたBaoらの研究も同様に，T年後(T=1，2，3)に開発者が貢献しているか否かを，プロジェクト参加直後1ヶ月間の貢献から予測モデルを構築している．これらの研究は，OSSの開発記録であるコミットやイシューへの対応など，プロジェクトでの全般的な活動を貢献とみなし，将来の単一時点での予測に留まっている．

2.2.3 開発者の離職予測に関する研究

Baoらは，商用企業を対象とし，プロジェクトに参加直後の6ヶ月間の月次レポートに基づき，1年後に離脱しているか否かを予測するモデルを提案している[16]．予測に寄与する特徴量には，月次レポートのタスク報告内容，労働時間の標準偏差，プロジェクト開発者での労働時間の標準偏差であった．この研究も同様に，月次レポートに基づく活動でありプロジェクトでの全般的な活動を貢献とみなしている．また，予測に関しても単一の時点である．

2.3 本研究の位置付け

図2.1に従来研究と本研究の違いを示す．従来研究では，コミットや月次レポートなどのプロジェクトへの貢献量に基づき，特定の期間以降に開発者の貢献があるか否かを予測する手法を提案している．従来手法は，将来的な継続を大まかに判断することには有効である一方で，連続する

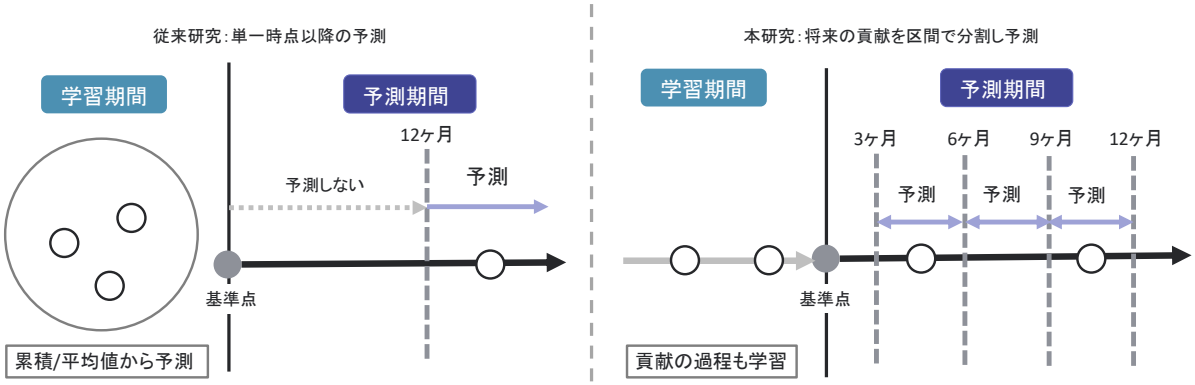

図 2.1: 従来研究と本研究の対比

各期間における貢献の有無を予測する手法は提案されていない．そのため，活動頻度の低下などによる開発者の離脱の兆候を把握したり，特定の期間において開発者の貢献の有無を予測し，タスクの負荷を分散したりすることが難しい．タスクが継続的に発生する開発プロセスにおいて，開発者の離脱の兆候や，特定の開発者に負荷が集中することを避けるためには，特定の期間以降ではなく，連続する期間において開発者の継続的な貢献を把握する必要がある．

OSS 開発では，自主的に貢献する活動に限らず，開発者間の協調作業として作業依頼に基づく活動も少なくない．本研究は，作業依頼に対しての応答が把握しやすく，開発者の貢献を捉えやすいコードレビューに着目し，開発者の継続的なレビュー依頼の承諾を予測し，継続的な貢献と捉える LTC 予測モデルを提案する．継続的な貢献とは各作業依頼に対して独立に判断するような単発の行動の繰り返しではなく，過去の活動やレビュー依頼を受けた際の状況に応じてレビュアーが依頼を承諾するか否かを選択する一連の意思決定プロセスである．このプロセスを，教師あり学習による予測では，ある時点までの累積値や平均値といった情報を用いて学習/予測を行うために，累積値や平均値に集約されるまでの過程や変化を捉えることが困難である．そのため，状態に応じた行動を選択する方策を逐次的に学習する強化学習の枠組みを活用することで，状態と行動の時系列的な遷移としてモデル化する．具体的には開発者の状態と行動の時系列的な遷移から，0 から 3ヶ月後，3 から 6ヶ月後，6 から 9ヶ月後，9 から 12ヶ月後にそれぞれ開発者がレビュー依頼を承諾するかどうかを予測するモデルを作成する．

第3章　継続的なLTC予測に向けた強化学習と逆強化学習

本研究では，OSS 開発者の継続的な貢献を，その時々の状況に応じてレビュー依頼を承諾するか否かを選択する一連の意思決定プロセスとして捉える．この時系列的な行動の軌跡に着目し，開発者の動的な行動パターンをモデル化するために，逆強化学習を用いて LTC 予測モデルを構築する. 本章では，逆強化学習の基盤となる強化学習について述べ，続いて逆強化学習および本研究での具体的な適用方法について述べる．

3.1　強化学習

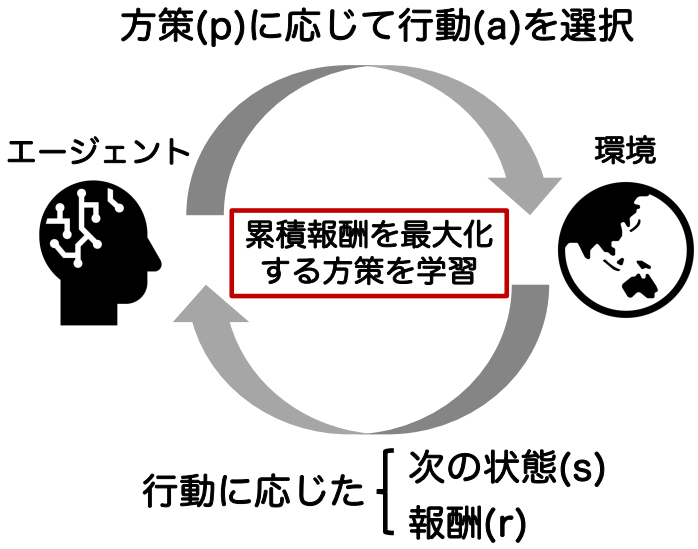

図 3.1: 強化学習の概略図

強化学習 (RL: Reinforcement Learning) は，ある環境に存在するエージェントが，その時点での状態を観測し，行動を選択することにより，行動や行動によって変化した状態に応じて，環境から得られる報酬の累積値を最大化するような行動の方策を学習する機械学習の 1 つの手法である．OSS 開発環境においては，レビュアーをエージェントとし，レビュー依頼に対してエージェントが承諾するか否かを逐次的に判断する意思決定プロセスに適用できる．

教師あり学習が正解データに基づき，学習を行うのに対し，強化学習は環境から得られる報酬を手がかりに最適な行動指針を自律的に探索するのが特徴である．数学的には，マルコフ決定過程として定式化され，次の 4 つの要素 S，A，T，R から成る．

- 状態集合 (S): エージェントが観測する環境の状態の集合．本研究ではレビュアーの過去の活動履歴や抱えているタスクの量などが該当
- 行動集合 (A): エージェントが選択可能な行動の集合．本研究では，アサインされたレビュー依頼に対して承諾するか否かが該当
- 遷移確率 (T): ある状態 s で行動 a を選択した際に，次の状態 s' へ遷移する確率
- 報酬関数 (R): 状態 s で行動 a をとった際に得られる即時報酬．エージェントがその行動に感じる価値を定量化したもの

表 3.1 は強化学習の基本的な構成要素を示す．

表 3.1: 強化学習の主な構成要素

用語	説明	OSS 適用例
エージェント	学習・意思決定を行う主体	レビュアー
環境	エージェントと相互作用する対象	OSS プロジェクト
状態	環境の現在状況	レビュアーの活動履歴など
行動	エージェントが取る選択肢	レビュー依頼を承諾するか否か
報酬	行動に応じて与えられるスカラー量	承諾なら高報酬
方策	エージェントの行動を決定する	レビューを引き受ける際の判断基準

強化学習において，報酬関数 $R(s, a)$ は最も重要な要素の一つであり，エージェントにとって何が望ましい行動であるかを示す指標となる．しかし，多くの実世界の問題では，多様な要因が複雑に作用しているため，事前に報酬関数を設計することは困難であり，不適切な報酬関数はエージェントを意図しない行動に導くことになる．

3.2 逆強化学習

強化学習において事前に報酬関数を設計することが困難である課題を解決するため，専門家の行動から，その行動を説明する報酬関数を逆算するアプローチとして逆強化学習が用いられる [17]．既知の報酬関数から最適な方策を学習する強化学習に対して，逆強化学習は既知の専門家の軌跡から，その行動を説明する報酬関数を推定する．

専門家の軌跡は，状態と行動を対として時系列に並べたものであり，専門家はある報酬関数（目的）を最大化するよう行動していると仮定し，その報酬関数を推定する．本研究では，レビュアーを専門家とみなし，レビュー依頼の承諾および非承諾を含む行動の軌跡から報酬関数を推定する．本研究への適用方法については，次に述べる．

3.3 本研究における適用方法

本研究では，OSS プロジェクトにおけるコードレビュー活動を，レビュアーがその時点の状態に応じて最適な行動を選択する，一連の意思決定プロセスとしてモデル化する．具体的には，レビュアーがその時点で置かれているプロジェクト内での立ち位置や，レビュアーの活動パターンおよび作業負荷などを捉えたものを状態として用いる．また，依頼されたレビューに対してレビュ

アーが承諾するか否かを行動とする．この際に，単にレビュー承諾するか否かのみでなく，対象となるレビュー依頼のタスク規模や，レビュー依頼に対する応答の早さといった側面も行動を表現する特徴量として用いる．

従来の教師あり学習を用いたLTC予測では，単一時点の特徴量の累積値や平均値といったスナップショットの情報を用いて学習を行う．この手法では，開発者のある期間における活動量を合計値や平均値として集約して計測することはできるが，活動頻度の変化などといった活動のパターンの変化を捉えることが難しい．これに対して，逆強化学習は時系列の活動を軌跡として扱い，各状態と行動から報酬関数を推定する．そのため，高負荷なタスクが重なる，レビューの応答時間が伸びるといった，単なる累積値や平均値では捉えることが難しかった活動量の変化の過程を考慮できる．時系列を考慮することにより，レビュー依頼数は多いが徐々に承諾率が低下しているレビュアーや，一時的に活動を休止した後に復帰するレビュアーなど，累積値や平均値では活動の変化が埋もれてしまうケースにおいても，開発者のレビュー承諾と拒否を判断することが可能であると考える．

第4章　LTC予測モデルの構築

4.1　データ収集・前処理

本研究では，コードレビュー管理システムから，変更提案の基本情報，レビュアー情報，レビューメッセージ，変更ファイル一覧などのレビュー活動に関するデータを収集する．収集したデータから自動化された bot アカウントと判断されるものをパターンマッチにより除外する．全てのデータをレビュー依頼時刻でソートし，時系列順に整列する．

逆強化学習では，将来の行動を予測する際に，過去の情報のみを用いるため，学習データと予測対象データを時間的に分離する．そのため，本研究では基準点を設定し，各基準点より過去の活動履歴を学習に用い，将来の一定期間におけるレビュー承諾の有無を予測する．

- 学習期間：
  スナップショットより過去の活動履歴を使用し，各ステップ時（スナップショット）から見て過去の活動履歴のみを使用する
- 評価期間：
  スナップショットより未来の a から bヶ月後（a-b m）の間で活動があるか否かを予測する

4.2　学習指標の作成

本研究では，LTC 予測モデルを構築するために，逆強化学習を用いて報酬関数を推定する．逆強化学習は，観測された行動の履歴から，その行動の評価基準を報酬関数として推定するものである．本研究で用いる逆強化学習では，基準点から指定期間内にレビュー依頼が承諾された場合を True，レビュー依頼が承諾されなかった場合を False とする二値変数として表現し，これを学習の入力データとする．この際の報酬関数は，開発者が継続してレビュー依頼を承諾しやすい状態にあるか否かを，過去の行動履歴に基づいて推定したものと解釈できる．

4.2.1　基準点の設定

レビュアーの貢献ごとにラベルを割り当てると，その数が膨大になり，学習時間の増加や，類似する状態に基づく学習が増加することで，過学習を引き起こすおそれがある．そこで，本研究では各月の末日を基準点とし，その月内の全活動に同一のラベルを割り当てる．本手法により，月単位での継続的な活動パターンを学習し，従来の単一時点以降の予測と比較して，特定の期間ごとに連続的に予測可能なモデルの構築を目指す．

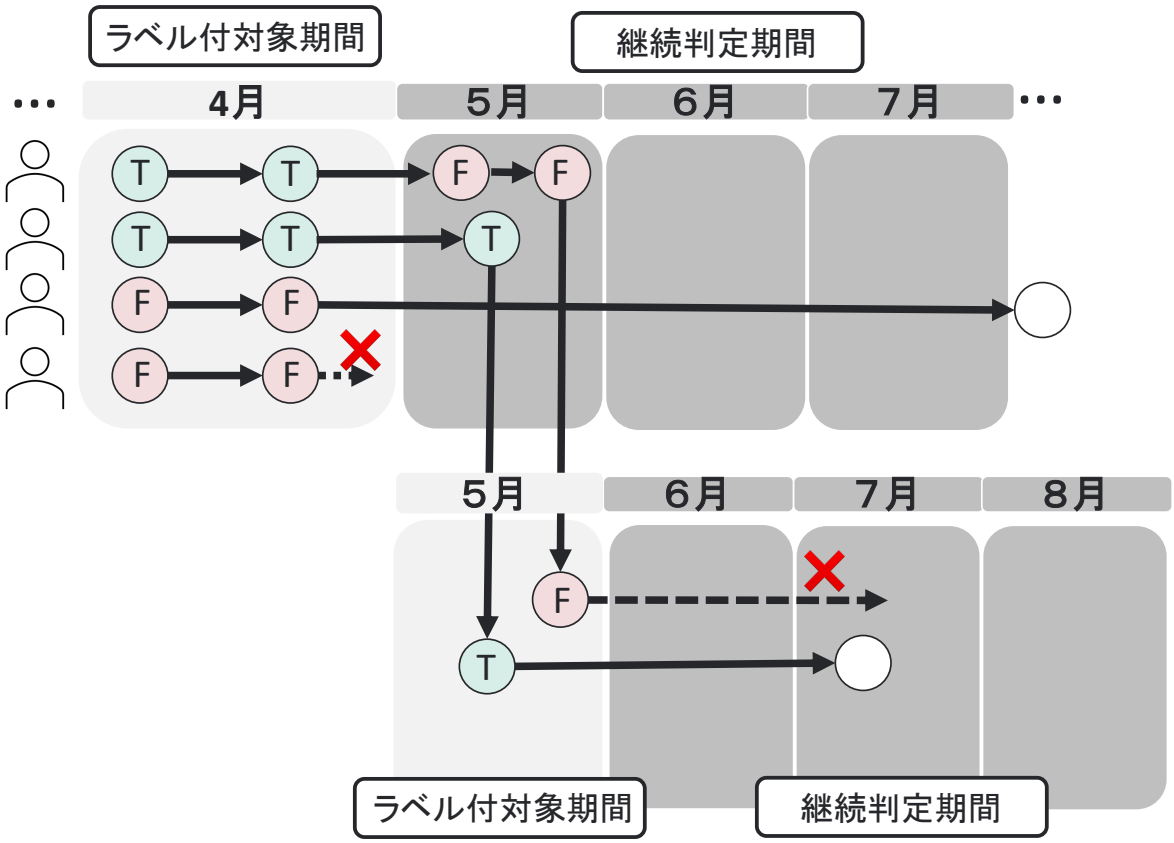

図 4.1: 指定期間が 0-3m であった時の例

4.2.2 レビュー承諾の有無を判定

学習指標の作成を図 4.1 に示し，具体的には次の手順で行う．図中の丸は，開発者の活動日を示しており，T，F はそれぞれ，各活動から見て指定期間後のレビュー承諾の有無を示す．

1. 各開発者の活動日から活動月を特定し，その月の最終日を基準点とする
   - 活動日が 2022 年 04 月 05 日，2022 年 04 月 14 日の場合 → 2022 年 04 月 30 日を基準点とする
2. 基準点から指定期間の間で将来の活動があるか調査（例：指定期間が 0-3m の場合）
   - 2022 年 04 月 30 日（基準点）から 2022 年 07 月 31 日（指定期間）にレビュー依頼があるか調査
3. 2 の期間でレビュー依頼を承諾するか否かを判定
   - レビュー承諾 → True，レビュー拒否 → False
4. その月の全活動に同じラベルを付与する
   - 4 月の全活動に T または F のラベルを付与（学習指標）

上記の 1~4 を学習期間内の各月で行い開発者の各活動にラベル付を行い学習指標を作成する．

4.2.3 学習指標の役割

本研究で用いる学習指標は，逆強化学習において単なる過去の事実を示すものではなく，レビュー依頼時の状態が将来のレビュー承諾にどのように影響するかを学習するための信号として機能する．具体的には，True の学習指標からは，レビュアーが継続して依頼を承諾する要因を学習し，False の学習指標からはレビュアーが依頼を承諾しない行動パターンを示す特徴を学習する．

4.3 状態・行動特徴量の生成

本研究では，逆強化学習を行うにあたり，表 4.1 に示す 10 個の状態特徴量と，4 個の行動特徴量を使用する．本研究では，将来のレビュー承諾の有無を予測するため，予測対象期間より前の時点までに観測された行動履歴を用いて特徴量を算出する．以降，この特徴量算出の基準となる時点を「予測時点」と呼ぶ．

表 4.1: 状態 (10 次元)/行動 (4 次元) 特徴量

	特徴量名	定義
状態特徴量	経験日数	初回活動から予測時点までの日数 [13]
	総レビュー依頼数	予測時点までに受けたレビュー依頼数 [18]
	総レビュー数	予測時点までに実施したレビュー数 [18]
	最近の活動頻度	1 日あたりの平均活動量 [19]
	平均活動間隔	活動間隔の平均日数 [16]
	月次活動変化率	直近 1 か月のレビュー活動量を前月と比較した活動量の変化率 [9]
	協力スコア	同じ開発者と共同でレビューした回数 [20]
	総承諾率	これまでのレビュー承諾率 [18]
	最近の承諾率	直近 10 件のレビュー依頼承諾数 [21]
	レビュー負荷	未完了レビュー数 [18]
行動特徴量	レビューファイル数	レビューするファイル数 [19]
	協力度	同じ開発者の変更提案をレビューした回数 [18]
	応答速度	レビューリクエストから応答までの日数の逆数 [19]
	レビュー規模	変更提案で変更があった総行数 [18]

4.4 モデル構造と学習フロー

本研究で学習に用いるニューラルネットワークモデルの概略図を 4.2 に示す．主に，エンコーダ，LSTM，デコーダから構成される．

エンコーダ

レビュアーの各活動を表す状態 (State) および行動 (Action) の 2 つの特徴量ベクトルを，それぞれ独立したエンコーダ (多層パーセプトロン) に入力する．これらのエンコーダは元のベクトル

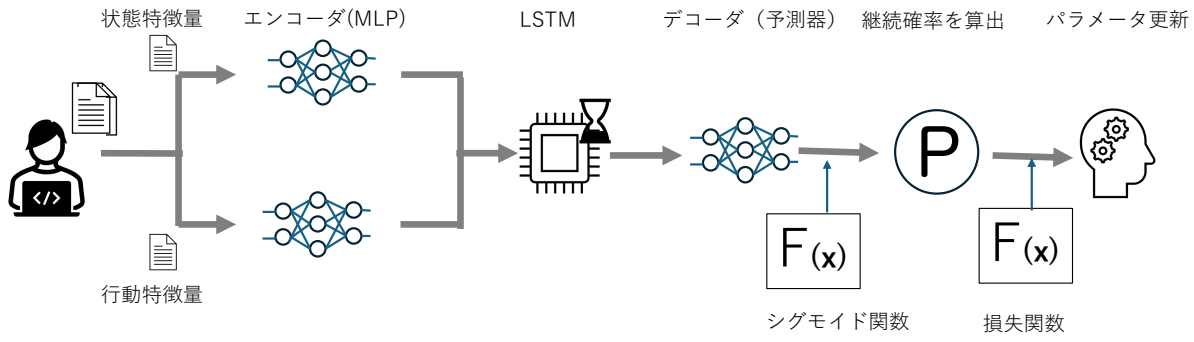

図 4.2: 学習から予測のフロー

よりもさらに表現力のある高次元のベクトルに変換する役割を持つ．

LSTM

各時点でエンコードされた状態ベクトルと行動ベクトルを連結し，時系列プロセッサである LSTM(Long Short-Term Memory) に入力する．LSTM は活動履歴を記憶し，時間的な依存関係を理解するため，単一の活動だけでなく，活動量が徐々に低下する動的なパターンも把握できる．

デコーダ

LSTM によりレビュアーの全活動が処理された後，LSTM で得られた時系列情報（特徴ベクトル）をもとに多層パーセプトロンで最終的な予測値を算出する．ここでは LSTM で処理されたレビュアーの全活動履歴を考慮した情報をデコードし，シグモイド関数により最終的な継続確率を算出する．

4.4.1 報酬関数

逆強化学習では，専門家の行動を説明する報酬関数を推定する．本研究ではレビュアーがレビュー依頼を承諾するか否かといった行動を，継続的な意思決定プロセスの結果として捉える．具体的には，レビュアーが状態 s において，レビュー依頼を承諾する，または承諾しないという行動 a を選択した際の評価を，報酬関数 $R(s, a; \theta)$ として表現し，ニューラルネットワークのパラメータ θ を学習する．

学習過程では，レビュー依頼を継続して承諾しているレビュアーの行動軌跡に対しては高い累積報酬が与えられるようにし，レビュー依頼が承諾されない行動軌跡に対しては低い累積報酬が与えられるように報酬関数を最適化する．ここで行動軌跡とは，レビュー依頼時点での状態と，それに対する行動の選択から構成される時系列データを指す．推定される報酬関数は，レビュアーが過去にどの程度継続してコードレビューに関与してきたのかを反映し，将来もレビュー依頼を承諾しやすい状態にあるか否かを数値として表現したものであると解釈できる．

継続確率 P は，推定された報酬関数に基づく行動軌跡の累積報酬をシグモイド関数 σ に入力することにより算出される．累積報酬が高いほど継続確率 P は高くなり，累積報酬が低いほど継続確率 P は低くなる．

学習時には 4.2 節で定義した学習指標を用い，予測確率 P と実際の結果との誤差を計算するために損失関数を用いる．この誤差が最小となるように，最適化アルゴリズム (Adam Optimizer) で反復的に学習を行うことにより，継続してレビュー承諾をするレビュアーの行動パターンを高く評価し，離脱者のパターンを低く評価する報酬関数を推定する．

4.4.2 損失関数による報酬関数の最適化

本研究では，推定された報酬関数に基づいて算出される継続確率と，実際に観測されたレビュー承諾の有無との誤差を最小化するために，損失関数を用いる．本研究では，将来の指定期間内にレビュー承諾があるか否かを予測する二値分類問題として定式化できる．従来研究では多くの開発者が LTC と呼ばれるまでに離脱することが知られている [4]. そのため，LTC 予測のデータセットは正例（LTC）と負例（LTC 以外の開発者）の間で学習データ数に不均衡が生じる．このような不均衡なデータに対しては適切な損失関数を用いることが重要である．

二値クロスエントロピー (BCE: Binary Cross-Entropy) は，二値分類において一般的に用いられる損失関数であり，モデルの予測確率と正解ラベルとの差を評価する．正解に近い予測ほど損失は小さく，誤った予測ほど損失が大きくなる性質を持つ．BCE は次の式で定義される [22].

二値クロスエントロピー：Binary Cross-Entropy (BCE)

$$L_{\mathrm{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(p_i) + (1-y_i)\log(1-p_i)\right]$$

ここで，N はデータセットのサンプル数，$y_i \in \{0,1\}$ は y_i の正解ラベル，モデルが予測したクラスの確率は $p_i \in [0,1]$ を示す.

$$p_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}$$

はシグモイド関数を表す.

不均衡データに対して BCE を用いた場合，容易に分類できる多数派クラスのサンプルの影響が大きくなり，少数派クラスに対応する重要な行動パターンの学習が不十分となる可能性がある．そこで，本研究では不均衡データによる問題を対処するため，BCE を拡張した損失関数である，Focal Loss を用いる．Focal Loss は予測が容易なデータの影響を抑え，予測が困難なデータをより重視して学習を行うことを目的としており，次の式で定義される．[22]

Focal Loss(FL)

$$L_{\mathrm{FL}} = -\frac{1}{N}\sum_{i=1}^{N}\Big(\alpha(1-p_i)^{\gamma} y_i \log(p_i) + (1-\alpha)p_i^{\gamma}(1-y_i)\log(1-p_i)\Big)$$

ここで，$\alpha \in [0, 1]$ はクラス不均衡を補正する重みパラメータを示し，$\gamma \geq 0$ は難しいサンプルを強調する焦点パラメータである．$\gamma = 0$ のときは，Focal Loss は BCE に一致し，レビュー承諾の有無を各サンプルの難易度を区別せずに学習する場合に相当する．

4.5 予測・評価

本研究では，学習時に評価期間の情報を使用することを防ぐため，学習期間と評価期間を分離するだけでなく，特徴量計算に使用可能なレビュー活動データの最大時点（Max-date）を設定している．具体的には，各予測時点において Max-date 以前のレビュー活動データから特徴量を算出し，その月末時点で継続の有無を示す教師信号（ラベル）を付与する．一方，Max-date 以降のレビュー活動データはラベル付の際のみ使用する．Max-date 以降のレビュー活動データを特徴量計算に使用し，将来時点のレビュー承諾の有無ラベルを参照すると，評価期間のデータを学習に用いることになり，適切な評価ができなくなる．そのため Max-date を設けることにより，モデルが評価期間の情報を事前に学習してしまうことを防止する．

さらに，最適化された報酬関数を用いて各特徴量に対する報酬を算出し，この報酬値にシグモイド関数を適用して 0 から 1 までの範囲に正規化することで，モデルの予測結果としての継続確率を得る．この確率 P が，次のレビュー依頼にレビュアーが承諾するか否かの予測値として用いる．

4.5.1 評価指標

本研究では，4 つの評価指標（適合率，再現率，F1 値，AUC-ROC）を用いる．本研究における二値分類の混同行列の構成要素（TP，FP，TN，FN）は以下のように定義される．

- **真陽性 (TP)**: レビュー依頼を承諾し，予測でも承諾すると正しく判定された件数
- **偽陽性 (FP)**: レビュー依頼を承諾していない（拒否または反応なし）が，予測では承諾すると誤って判定された件数
- **真陰性 (TN)**: レビュー依頼を承諾しておらず，予測でも承諾しないと正しく判定された件数
- **偽陰性 (FN)**: レビュー依頼を承諾したが，予測では承諾しないと誤って判定された件数

これらの値を用いて，各評価指標は次式で定義される．

$$\text{適合率 (Precision)} = \frac{TP}{TP + FP}, \quad \text{再現率 (Recall)} = \frac{TP}{TP + FN}$$

$$\text{F1 値 (F1 Score)} = \frac{2 \cdot \text{適合率} \cdot \text{再現率}}{\text{適合率} + \text{再現率}}$$

AUC-ROC は閾値を変化させたときの ROC 曲線（横軸に偽陽性率，縦軸に真陽性率）下面積であり，閾値に依存せず，レビュー依頼を承諾するレビュアーと承諾しないレビュアーを区別するモデルの基本的な能力を総合的に評価する．ROC 曲線に用いる真陽性率（TPR）と偽陽性率（FPR）は次式で定義する．

$$\text{真陽性率 (TPR)} = \frac{TP}{TP + FN}, \quad \text{偽陽性率 (FPR)} = \frac{FP}{FP + TN}$$

4.5.2 クロス評価

従来研究では，予測時点以降の任意の時点（例えば，開発者が OSS プロジェクトに参加して 1 年後）以降における貢献の有無を予測していた．本研究では，未来の任意の区間での貢献の有無を予測し，この区間をスライドさせながら予測することで，どの学習方法が将来の活動の予測に最も寄与するかを明らかにする．また，各学習期間において，報酬を算出するために参照する指定期間も，同様にスライドする．具体的には，レビュー依頼を受けた時点の月の末日を基準に，学習時に参照する指定期間をスライドさせた 4 パターン（0 から 3 か月，3 から 6 か月，6 から 9 か月，9 から 12 か月）のモデルに対して，学習期間 $\leq$ 評価期間となるように評価期間をスライドさせた計 10 パターンで評価する．以降の実験結果および考察において，基準点から X ヶ月後から Y ヶ月後の期間を表す際に「X-Ym」という表記を用いる（例：3ヶ月後から 6ヶ月後の期間は 3-6m とする）．また，特定の期間（例：3-6m）の貢献有無を学習したモデルを「3-6m モデル」のように表記する．

第5章　ケーススタディ

本章では，逆強化学習に基づく提案モデル（以降，IRL）と，比較手法としてランダムフォレスト（以降，RF）を用いて評価を行う．

5.1　データセット

本研究では，コードレビュー管理システムである Gerrit[1]による詳細なレビュー履歴が利用可能かつ大規模で十分なデータが蓄積されていることから，仮想マシンインスタンスのライフサイクル管理を行う OpenStack の中核サービス OpenStack/Nova を対象とした．

Gerrit REST API を用いて，レビュー活動に関するデータを収集し，自動化された bot アカウント（bot，cl，jenkins，zuul）と判断されるものを，レビュアーのメールアドレスのパターンマッチにより除外した．全てのデータをレビュー依頼時刻でソートし，時系列順に整列した．表 5.1 は，分析対象プロジェクトの統計データを示す．

表 5.1: データセットの概要

項目	値
対象期間	2021 年 01 月 01 日 〜 2024 年 01 月 01 日
学習期間	2021 年 01 月 01 日 〜 2023 年 01 月 01 日
評価期間	2023 年 01 月 01 日 〜 2024 年 01 月 01 日
総レビュー依頼数	約 5,473 件
学習時レビュアー数	約 70 人
評価時レビュアー数	約 60 人

5.2　RQ1: 逆強化学習に基づく提案モデルは，コードレビューにおけるレビュー承諾をどの程度予測できるか

RQ1 では，構築した IRL のモデルが学習対象の期間における開発の行動パターンを適切に学習し予測できるのか，また過学習になっていないかを検証することを目的としている．設定した学習期間のうち，最も短期的な予測（0-3m）と最も長期的な予測（9-12m）の中間に位置し，モデルの標準的な挙動を確認するために 3-6m モデルを代表例として取り上げ，評価する．

図 5.1 は，3-6ヶ月後に貢献があるかをラベル付し学習した 3-6m モデルにおける，学習期間での予測の精度と，評価期間での予測の精度を示している．具体的には，Precision，Recall，F1 Score，

[1]https://www.gerritcodereview.com/

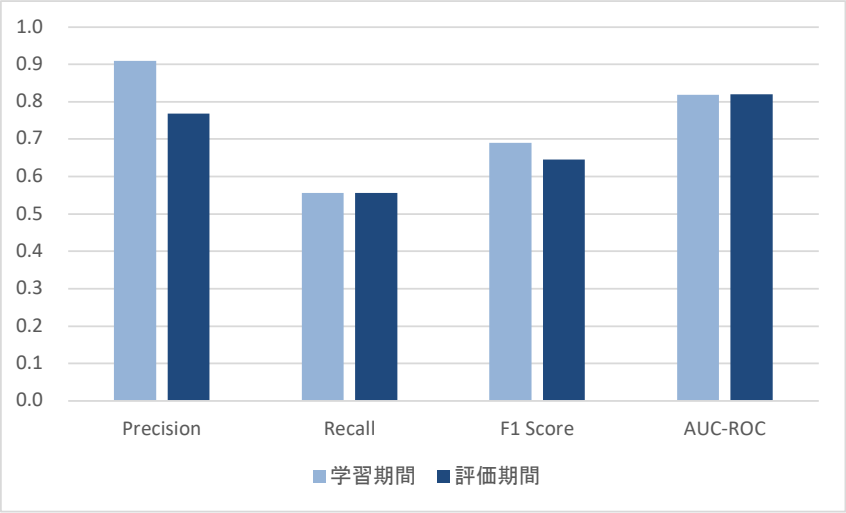

図 5.1: 学習・評価期間における予測結果

AUC-ROC の 4 つの評価指標を用いており，各指標のグラフの左側が学習期間，右側が評価期間の結果を示している．

学習期間と評価期間の両期間における各指標の値を比較すると，大きな乖離が見られず同程度の精度を維持していることが確認できる．この結果は，提案モデルが学習データに過剰に適合しておらず，学習データに含まれない将来の期間（未知の期間）におけるレビュアーの行動予測に対しても，汎化性能を有していることを示唆する．また，評価期間の予測における AUC-ROC が 0.820 であり，提案モデルがレビュー依頼を承諾する貢献者と離脱者を効果的に区別できることを示している．

5.3 RQ2: 学習方法や評価期間の長さに応じて，予測モデルの精度が異なるのか

RQ2 では，学習期間および評価期間の長さが予測精度に与える影響を明らかにするため，学習期間 $\leq$ 評価期間の条件下で 10 パターンのクロス評価を実施した．具体的には，4 つの学習期間（0-3m，3-6m，6-9m，9-12m）に対して，それぞれの学習期間以上の期間の評価期間で評価した．図 5.2 と図 5.3 は，それぞれ IRL と RF における学習期間と評価期間の組み合わせごとの精度をヒートマップで示している．横軸は学習期間，縦軸は評価期間を表す．

本実験では，学習データの期間設定と評価期間の設定が同一である場合に，開発者の貢献傾向といったデータの性質が最も類似していると考えられるため，予測精度が高くなると予想される．しかし，結果として IRL の AUC-ROC に着目すると，学習期間と同一の評価期間よりも，さらに後の期間（例：学習期間 0-3m で評価期間 6-9m）で 0.910 となり，高い精度を示した．

一方，RF の AUC-ROC は，学習期間に関わらず評価期間 6-9m において一貫して高精度（0.900 程度）を示した．しかし，評価期間が 0-3m，3-6m，9-12m のパターンでは，IRL が RF の AUC-ROC を上回った．また，評価期間 9-12m では，評価期間が 6-9m の場合と比較し，RF の AUC-ROC は大きく低下した．

これらの結果から，評価期間の長さに関しては，両手法とも評価期間 6-9m で最も高精度となり，9-12m で精度が低下した．学習期間の長さに関しては，IRL では学習期間 3-6m のモデルが評価期間に関わらず平均的に高い精度を示し，学習期間 9-12m のモデルでは精度が低下した．一方，RF では評価期間 6-9m において学習期間に関わらず安定した高精度を示した．また，IRL は評価期間 6-9m を除く大部分のパターンで RF より高精度であり，特に評価期間 9-12m では RF が大きく精度低下する一方で IRL は相対的に高精度を維持した．

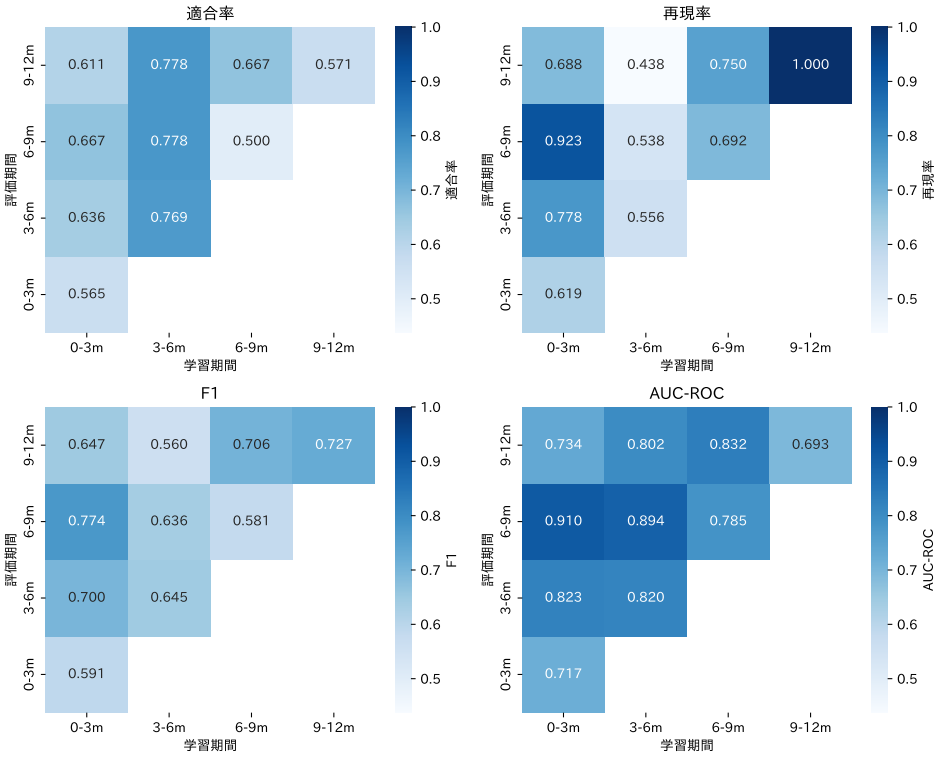

図 5.2: IRL の各モデルのクロス評価

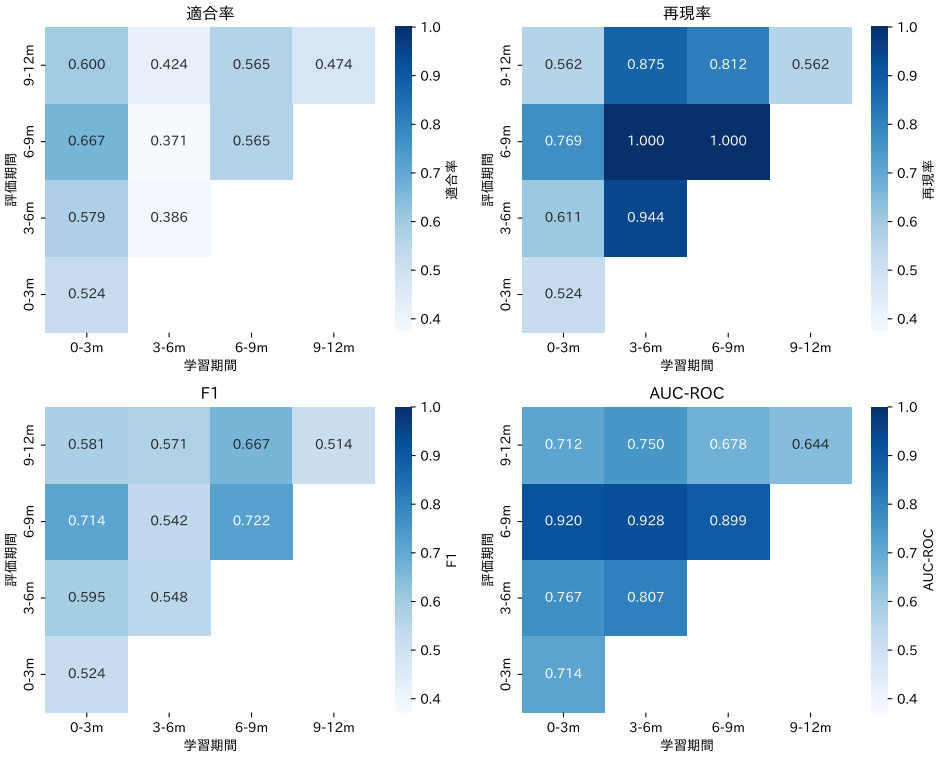

図 5.3: RF の各モデルのクロス評価

5.4 RQ3: 推定された報酬関数において，LTC の継続的なレビュー承諾に寄与する特徴量は何か

RQ3 では，IRL および RF を用いて特徴量の重要度分析を行い，継続的なレビュー承諾に寄与する要因を明らかにする．比較手法である RF モデルの重要度分析に関しては，Gini 係数に基づく重要度の分析を行なった．これは，決定木の不純度の減少量を基準としており，その特徴量が分類精度向上にどの程度寄与するのかを示す指標である．

また，提案手法である IRL の重要度分析に関しては，ニューラルネットワークによって近似された報酬関数 $R(s, a)$ を解析対象とし分析を行なった．ここでは，入力する特徴量の変化が，継続確率である報酬値に与える影響を分析するために，勾配ベースの重要度分析を行なった．具体的には，学習済みモデルにおいて入力特徴量に対する報酬関数の偏微分係数（勾配）を算出する．これは，ある特徴量の値を微小に変化させた際，出力である報酬値がどの程度変動するかを定量化するものである．例えば，ある特徴量の入力値を僅かに増加させた結果，出力される報酬値が増加すればその特徴量は継続を促進する，正の影響を与えていると解釈できる．逆に，入力の増加に伴って報酬値が減少すればそれは継続を阻害する，負の影響を与えていることになる．また，値が 0 に近い場合はその特徴量が予測（報酬値の増減）に与える影響が小さいことを示す．

図 5.4 は IRL，図 5.5 は RF における各特徴量の重要度をそれぞれモデル別で分析した結果である．IRL では，レビュアーの貢献を予測する状態特徴量として総レビュー数が，行動特徴量としては協力度が強い正の影響を与えていることが明らかになった．また，平均活動間隔，レビュー規模，総承諾率は強い負の影響を与えていることが明らかになった．さらに，予測する期間が長くなるにつれ，重要とされる特徴量が総レビュー数から協力度に変化した．次に RF では，主に総レビュー依頼数が重要であり，9-12m のモデルにおいては，これまでのレビューの承諾率である総承諾率やレビュー負荷が重要であることがわかった．

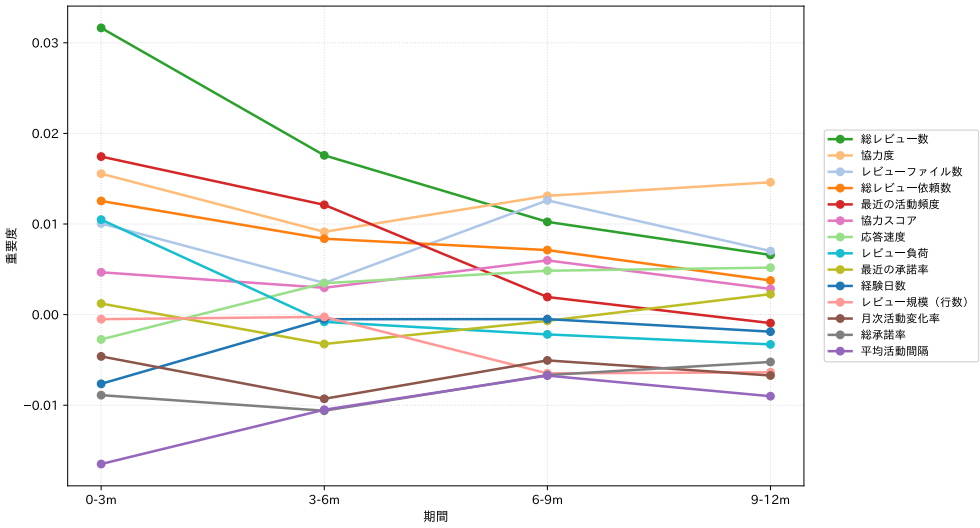

図 5.4: 各特徴量の予測における重要度 (IRL)

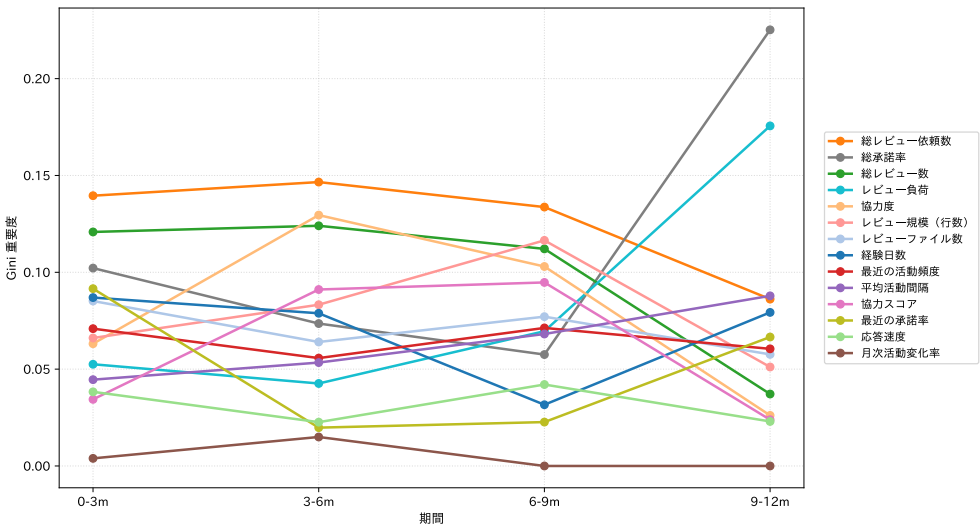

図 5.5: 各特徴量の予測における重要度 (RF)

第6章　考察

本章では，RQ1〜RQ3 の結果を踏まえ，時系列データの考慮が予測に与える影響，短期予測と長期予測における特徴量の重要度の違い，IRL と RF で予測が不一致の開発者について考察する．

6.1　時系列データの考慮が予測に与える影響

図 5.2，図 5.3 から，IRL と RF で評価期間によって予測精度が異なることが明らかになった．RF は IRL と比較して，AUC-ROC の最大値が 0.928 であり，IRL の最大値である 0.910 より高くなった．しかし，IRL は RF と比較し，評価期間が 6-9m の場合を除いたすべてのパターンで AUC-ROC が高くなることが明らかになった．

この精度差の要因を明らかにするため，評価期間ごとの開発者の行動パターンを分析した．図 6.1 は，評価期間ごとの承諾率の階級の構成比を示している．

評価期間内のレビュアーの承諾率の分布を確認すると，期間ごとにレビュアーの承諾率の分布が大きく変化している．例えば，3-6m では承諾率 0–10%の層が 38.1%と最多であるのに対し，9-12m では 70–100%の層が 55.6%と過半数を占めることがわかる．

このように，低承諾が多数派の期間から高承諾が多数派の期間へと分布が移動しており，単一時点の特徴量だけでは捉えることが困難である特徴量の変化が存在することが考えられる．時系列的な変化を軌跡として学習する IRL は，こうした分布の移動を反映しやすく，複数期間で安定的に精度を確保できたと考える．また，IRL において，学習期間が 3-6m であるモデルは，他の学習期間のモデルと比較して AUC-ROC が平均的に高くなることがわかる．これは分析対象とする Nova プロジェクトが 6ヶ月ごとにリリースしていることが一要因として考えられる．

6.2　短期予測と長期予測における特徴量の違い

図 5.4 に示す通り，IRL において各モデルにおいて予測に寄与する特徴量の重要度の変化が確認された．短期 (0-3m モデル) では総レビュー依頼数やレビュー数などの活動量が予測に大きく寄与したが，期間が進むにつれてその影響が減少し，長期 (9-12m モデル) では協力度や活動間隔やレビューファイル数といった活動量以外の要因の寄与が高まった．これは，0-3m のモデルで，評価期間が 9-12m にまで離れると，活動量を重視した予測を行うが，活動量以外の特徴量も重要となるために，予測精度が低下する原因になると考えられる．

図 6.2 は，評価期間ごとのレビュー依頼数，承諾率，応答速度の分布を示している．レビュー依頼数の中央値は各期間を通じて大きな変化は見られない．一方で，レビュー承諾率の中央値に関しては，評価期間により変化が見られ，0-3m では 0.500 であったのに対し，9-12m では 0.773 に上昇しており，評価期間が長くなるにつれ，承諾率が上昇する傾向が見られた．応答速度に関し

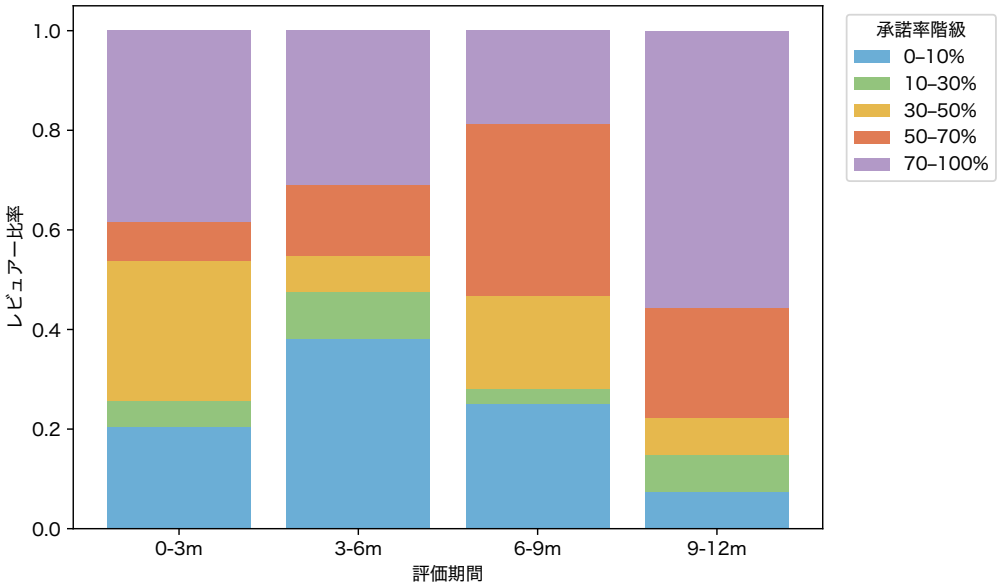

図 6.1: 承諾率階級の構成比の推移（レビュアー単位）

ては，中央値は 0-3m では 7.0 日であったのに対し，9-12m では 4.5 日まで短縮している．これらの結果は，活動量の規模が同程度であっても，承諾率や応答速度といった側面が期間とともに変化していることを示し，短期間の予測では活動量による影響が強く，長期間になるにつれて単純な活動量のみでなく，協力度や応答速度といった要因の影響が高くなることを示唆する．

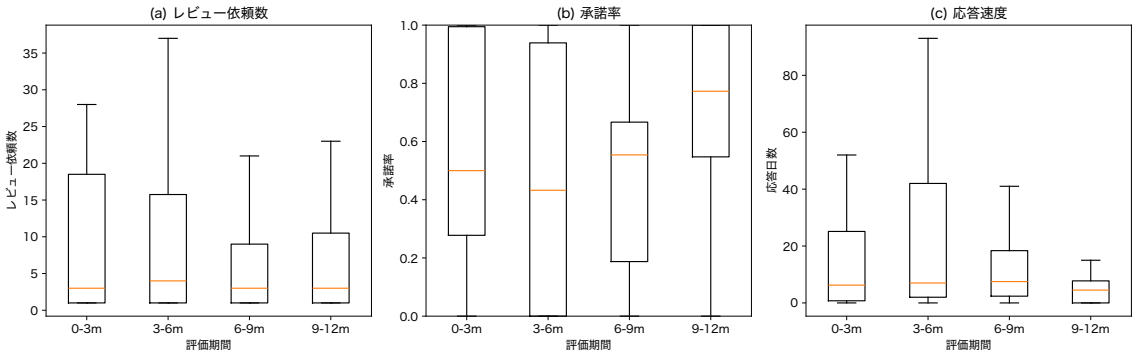

図 6.2: 評価期間別のレビュー依頼数・承諾率・応答速度の分布（レビュアー単位）

さらに，レビュー規模，先月の活動回数と比較した活動回数の上昇率である月次活動変化率は，一貫して負の影響を示した．また，レビュー負荷も負の値であるモデルが多いことから，レビュー規模や回数が過度に増加するとレビュアーへの負担が増加し，離脱の要因となることが示唆される．平均活動間隔においても一貫して負の影響を示し，活動間隔が増加することもレビュアーの離脱の要因となることが示唆される．

一方で，RF に関しては 図 5.5 に示す通り，モデルによって重要とされる特徴量の変化は見られたものの，どのモデルにおいても，総レビュー依頼数，総承諾率といったレビュー活動の積み重ねに関する特徴量が最も重要な特徴量であった．RF においては，時系列的なタスクのつながりを持たないために，月次活動変化率の重要度が低くなっていることが考えられる．

IRL においては，報酬を増加させることが直接タスクの承諾確率の向上につながるため，RF と比較してタスクの承諾確率を向上させるためにレビュアーに対してどのような行動を起こすと良いのかを判断しやすく，プロジェクトのマネジメントに有効であると考える．本分析の結果は，レビュー規模や依頼頻度が過度に増加しないよう配慮し，レビュアーの活動間隔が長期化する前に適切なタスクを割り当てることが，レビュアーの継続的な活動を促進する上で重要であると示唆する．

6.3 IRL/RF において予測不一致の開発者について

図 6.3 は，RQ2 における 10 パターン学習・評価期間の組み合わせでの，IRL と RF の予測結果のベン図を示している．各パターンで 39~60 名のレビュアーを対象に予測を行い，合計で 452 件の予測結果を得た．IRL と RF の結果を比較すると，両手法とも正解した件数は 216 件であり，全体の 47.8%を占めた．一方で，IRL のみが正解した件数は 112 件（24.8%），RF のみが正解した件数は 72 件（15.9%）であった．次に提案手法のみが正解した事例について述べる．

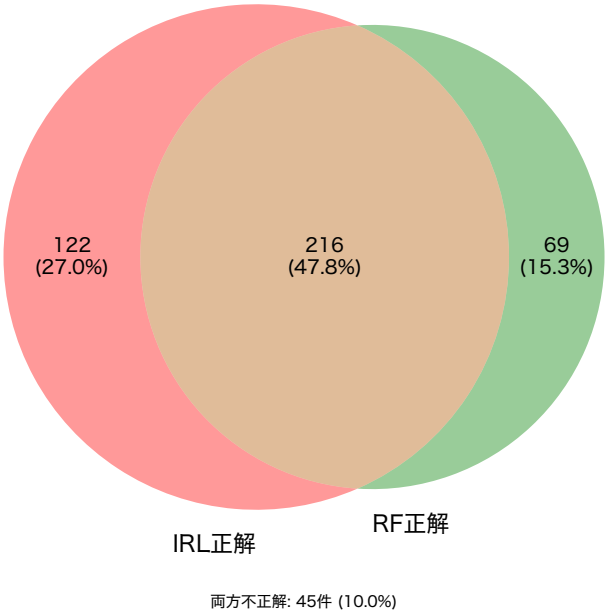

図 6.3: IRL/RF における予測結果の正解集合の関係（全 452 件）

IRL のみが正解したケースの特徴

IRL のみが正解した 122 件の事例について考察する．図 6.4 は，真のラベル (承諾/非承諾) ごとに，レビュー依頼数と承諾率の散布図において，RF と IRL を比較した際に両モデルとも正解/不正解，IRL のみ正解，RF のみ正解の 4 つのグループに分類し，プロットしたものである．左図がレビューを承諾したレビュアー，右図がレビューを承諾しなかったレビュアーの分布を示している．特に，レビュー承諾しなかったレビュアー (6.4 右図) に着目すると，レビュー依頼が中央値よ

り多く，承諾率が中央値より低い（右下の領域）で IRL のみの正解が比較的多い．これは，RF がレビュー依頼数の多さに引きずられて「レビュー承諾」と誤って予測した一方で，IRL は承諾率の低さや活動パターンの時系列的な変化を捉えることで，正しく「レビュー非承諾」と予測できたためであると考える．

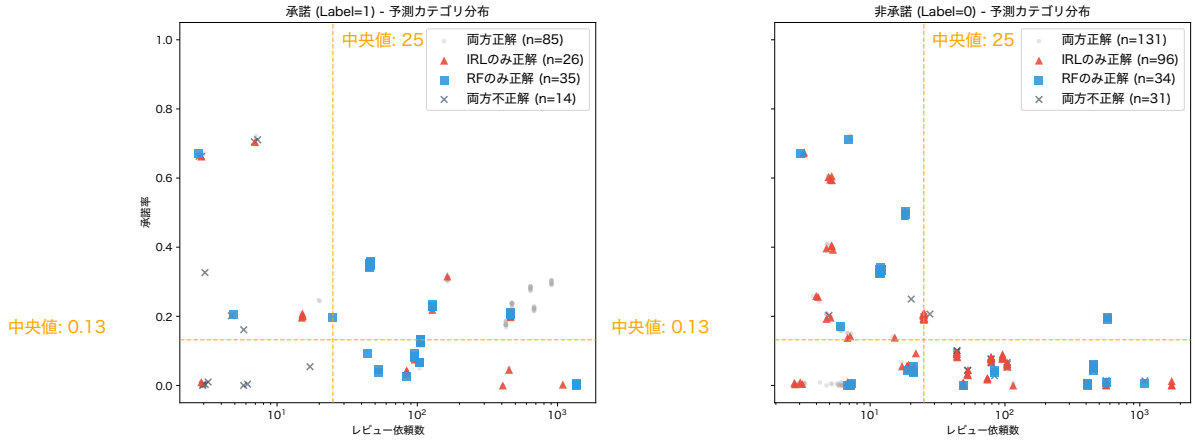

図 6.4: 真のラベル（承諾/非承諾）別のレビュー依頼数と承諾率に基づく予測結果の分布

活動パターンの変動があったレビュアー

4 つの全ての評価期間 (0-3m，3-6m，6-9m，9-12m) に存在する 39 名のレビュアーについて，各期間での活動パターンを分析した．

これらのレビュアーを活動パターンで分類すると，一貫承諾型，一貫非承諾型，変動型の 3 つに分類された．

- **一貫承諾型**：全期間で承諾を続けるレビュアー（11 名，28.2%）
- **一貫非承諾型**：全期間で承諾しないレビュアー（15 名，38.5%）
- **変動型**：期間によって承諾の有無が変動するレビュアー（13 名，33.3%）

図 6.5 は，これら 3 つの活動パターン分類別の IRL と RF の予測正解率を示している．変動型の内訳として，特定の 1 期間のみ承諾するパターンが 9 名と最も多く，そのうち 4 名が 3-6m のみ承諾するパターンであった．これは，Nova プロジェクトの 6ヶ月リリースサイクルに対応する期間であり，リリースに関連した一時的な貢献の存在が示唆される．また，0-3m で承諾した後に 3-6m，6-9m で離脱（非承諾）し，9-12m に復帰（承諾）するパターンも 3 名確認され，こうした変動パターンの存在は期間ごとの予測が必要であることを示唆している．

変動型レビュアーと従来手法の限界

従来の LTC 予測研究では，予測時点から 1 年後や 3 年後のような単一時点以降に貢献があるか否かを予測する．この枠組みを本研究のデータに適用し，最終期間（9-12m）における活動の有無

で全期間の判定を行うとすると，一貫型のレビュアーについては正しく判定できるが，変動型のレビュアーについては誤判定が生じる可能性がある．

具体的には，非承諾/承諾を 0/1 のバイナリで各期間の予測を表現すると，パターン 1001 のレビュアーは，9-12m に活動があるため従来手法では LTC と判定されるが，実際には 3-6m，6-9m の 2 期間は活動がなく，この期間にレビュー依頼をしても承諾は期待できない．逆に，パターン 0100 のレビュアーは，9-12m に活動がないため従来手法では非 LTC と判定されるが，3-6m 期間には活動可能であり，この期間に限定したタスク割り当てが可能である．

このように，従来の単一時点予測では，中間期間における非承諾/承諾を捉えることができない．そのため，連続的な予測により，このような変動型の活動パターンを区間ごとに予測することで，タスク配分の最適化やレビュアーのリソース管理に貢献し得ると考える．これらの分析から，レビュアーの約 3 分の 1 は期間によって活動の有無が変化する変動型のパターンを持ち，この変動型レビュアーの存在は，連続的な区間予測の必要性を示唆する．

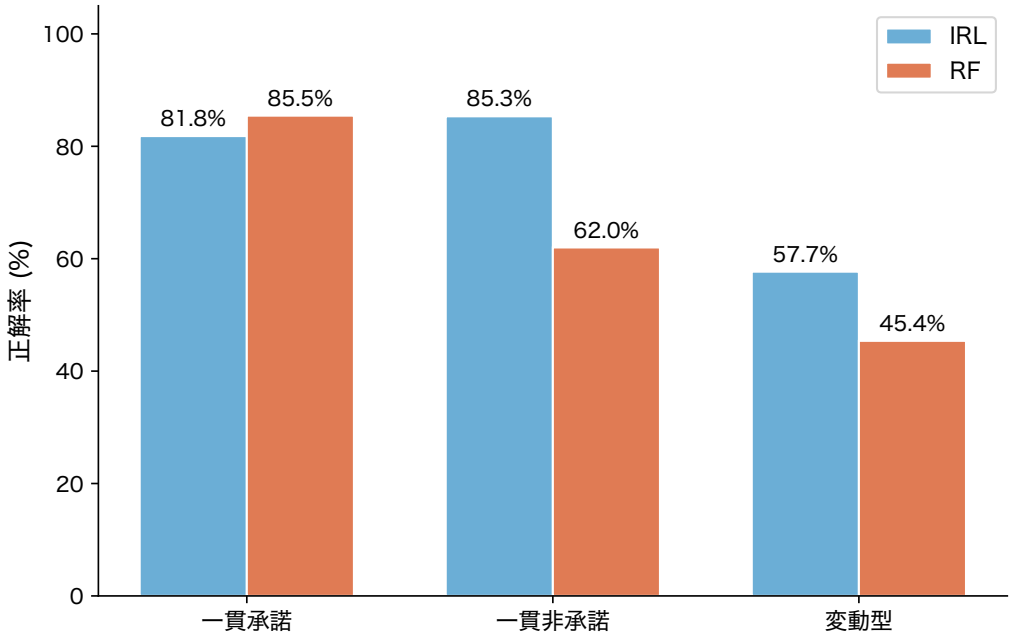

図 6.5: 活動パターン分類別の IRL/RF 正解率

第7章　妥当性の脅威

7.1　内的妥当性の脅威

本研究では，レビュアーの状態と行動を表すために，それぞれ10次元と4次元の特徴量を定義したが，これらの特徴量がレビュアーのタスク承諾の意思決定の全てを網羅しているわけではない．しかし，本研究では，レビュー活動の代表的な特徴量である，活動の量や質，社会性といった多面的な視点をカバーしているため，OSSレビュアーの多様な行動の傾向を理解するための一指標として有効であると考える．また，逆強化学習は観測されたデータからその背後にある行動原理（報酬関数）を推定するものであり，未観測のデータに対してもある程度の頑健性を持つことが考えられる．

7.2　外的妥当性の脅威

本研究では，OpenStack/Novaという単一プロジェクトを対象としている．OpenStack/Novaは独自の開発形態やレビュープロセスを持っており，本研究で得られた結果が，他のドメインや開発プロセスを持つプロジェクト（例：GitHubを使用しているプロジェクトなど）においては，今回の結果とは異なる新たな知見が得られる可能性がある．しかし，OpenStack/Novaは，大規模かつ長期間運用されている代表的なプロジェクトであるため，OSS開発における継続的な貢献パターンの一般的プロジェクトの一例として有効な知見が得られると考える．

また．分析対象とした期間（2021年〜2024年）において，対象プロジェクトであるNovaプロジェクトはプロジェクトを立ち上げてから十分な期間が経過しており，成熟したプロジェクトを対象にしている．そのため，プロジェクトの立ち上げ期や，成長のフェーズが異なる場合には，重要となる特徴量が異なる可能性がある．しかし，成熟したプロジェクトにおいても開発者の入れ替わりが発生しており，新規開発者が多数存在する．そのため，新規開発者がどのようにコミュニティに定着していくのかを学習・予測するといった流れをとっているため，成熟期のプロジェクトを対象にすることで，長期的な貢献者と新規の開発者の両者を学習できるため妥当であると考える．

第8章　おわりに

本研究では，OSS 開発における LTC の予測手法として，逆強化学習に基づくアプローチを提案した．従来研究では，プロジェクト参加後の一定期間以降に貢献があるか否かを予測していたが，本研究では，コードレビュープロセスに着目し，逆強化学習を用いることで，連続する各期間におけるレビュー承諾を予測するモデルを構築した．

具体的には，コードレビューにおけるレビュー依頼の承諾を継続的な貢献と捉え，開発者の状態と行動を時系列の軌跡としてモデル化し，報酬関数を推定することで予測を行った．OpenStack/Nova プロジェクトを対象としたケーススタディにより，提案手法が従来の RF と比較して，多くの評価パターンにおいて高い予測精度を達成することを示した．

本研究の主要な成果として，時系列的な開発者の行動パターンの変化を考慮することで，従来手法では捉えることが困難であった活動量の変化や承諾率の推移といった動的な特性を予測に反映できることを明らかにした．特に，評価期間が 6-9m の場合を除く多くのパターンにおいて，従来手法を上回る予測精度を達成した．また，予測する期間の長さに応じて重要となる特徴量が変化することを示した．短期予測ではレビュー依頼数などの活動量が重視される一方，長期予測では協力度や応答速度などの活動量以外の要因の重要性が高まることが明らかになった．この知見は，期間に応じた適切なタスク配分戦略の構築に貢献すると考えられる．さらに，約 3 分の 1 の開発者が期間によって活動の有無が変動する変動型のパターンを持つことを明らかにした．従来の単一時点予測では，こうした開発者の中間期間における離脱や復帰を捉えることができないため，連続的な区間予測の必要性を示した．

本研究の結果は，プロジェクトマネージャーが開発者の継続的な貢献を予測し，適切なタスク配分やサポートを行う上で有用な知見を提供する．特に，逆強化学習により推定された報酬関数は，どのような状態や行動が開発者の継続的な貢献を促進するかを示すため，プロジェクトのマネジメントにおいて実用的な指針となる．

謝 辞

本研究を進めるにあたって，多くの方々に，御指導，御協力，御支援を賜りました．ここにお世話になった方々への感謝の意を記させていただきます．

はじめに，指導教員である和歌山大学システム工学部伊原彰紀准教授に対し，厚く御礼申し上げます．研究室配属以来，研究の方向性や論文の執筆，プレゼンテーションの方法など，多くの時間を割いて御指導いただきました．そして，数多くの研究発表の機会を与えていただき，様々な研究者との交流をさせていただくことによって，自分の視野を広げることができました．先生の御尽力に敬意を表し，心より感謝いたします．

また，和歌山大学ソーシャルソフトウェア工学研究室の方々には，普段から多大な御協力，御助言をいただきました．特に，ソーシャルソフトウェア工学研究室の同期には，研究のことだけでなく大学生活でも大変お世話になりました．おかげで充実した研究活動，大学生活を送ることができました．心より深く感謝いたします．

最後に，日頃から暖かく見守っていただきました家族に対し，心より深く感謝いたします．

参 考 文 献

[1] Yunwen Ye and Kouichi Kishida. Toward an understanding of the motivation of open source software developers. In *Proceedings of the 25th International Conference on Software Engineering*, ICSE'03, pp. 419–429, 2003.

[2] Eric Raymond. The cathedral and the bazaar. *Knowledge, Technology & Policy*, Vol. 12, No. 3, pp. 23–49, 1999.

[3] James Howison, Keisuke Inoue, and Kevin Crowston. Social dynamics of free and open source team communications. In *Proceedings of the IFIP International Conference on Open Source Systems*, IFIP'06, pp. 319–330, 2006.

[4] Amanda Lee and Jeffrey C Carver. Are one-time contributors different? a comparison to core and periphery developers in floss repositories. In *Proceedings of the ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, ESEM'17, pp. 1–10, 2017.

[5] Jailton Coelho and Marco Tulio Valente. Why modern open source projects fail. In *Proceedings of the 11th Joint meeting on foundations of software engineering*, ESEC/FSE'17, pp. 186–196, 2017.

[6] Vijaya Kumar Eluri, Thomas A Mazzuchi, and Shahram Sarkani. Predicting long-time contributors for github projects using machine learning. *Information and Software Technology*, Vol. 138, pp. 1–13, 2021.

[7] Karl Fogel. *Producing open source software: How to run a successful free software project.* " O'Reilly Media, Inc.", Sebastopol, CA, 2005.

[8] Minghui Zhou and Audris Mockus. What make long term contributors: Willingness and opportunity in oss community. In *Proceedings of the 34th International Conference on Software Engineering*, ICSE'12, pp. 518–528, 2012.

[9] Lingfeng Bao, Xin Xia, David Lo, and Gail C. Murphy. A large scale study of long-time contributor prediction for github projects. *IEEE Transactions on Software Engineering*, Vol. 47, pp. 1277–1298, 2021.

[10] Md Ahasanuzzaman, Gustavo A Oliva, and Ahmed E Hassan. Predicting long time contributors with knowledge units of programming languages: an empirical study. *Empirical Software Engineering*, Vol. 30, No. 3, p. 99, 2025.

[11] Chorng-Guang Wu, James H Gerlach, and Clifford E Young. An empirical analysis of open source software developers'motivations and continuance intentions. *Information & Management*, Vol. 44, No. 3, pp. 253–262, 2007.

[12] Chris Jensen and Walt Scacchi. Role migration and advancement processes in ossd projects: A comparative case study. In *Proceedings of the 29th International Conference on Software Engineering*, ICSE'07, pp. 364–374, 2007.

[13] Christian Bird, Alex Gourley, Prem Devanbu, Anand Swaminathan, and Greta Hsu. Open borders? immigration in open source projects. In *Proceedings of 4th International Workshop on Mining Software Repositories*, MSR'07, pp. 6–6, 2007.

[14] Igor Steinmacher, Igor Wiese, Ana Paula Chaves, and Marco Aurélio Gerosa. Why do newcomers abandon open source software projects? In *Proceedings of the 6th International Workshop on Cooperative and Human Aspects of Software Engineering*, CHASE'13, pp. 25–32, 2013.

[15] Akinori Ihara, Yasutaka Kamei, Masao Ohira, Ahmed E Hassan, Naoyasu Ubayashi, and Ken-ichi Matsumoto. Early identification of future committers in open source software projects. In *Proceedings of the 14th International Conference on Quality Software*, QSIC'14, pp. 47–56, 2014.

[16] Lingfeng Bao, Zhenchang Xing, Xin Xia, David Lo, and Shanping Li. Who will leave the company?: a large-scale industry study of developer turnover by mining monthly work report. In *Proceedings of the 14th International Conference on Mining Software Repositories*, MSR'17, pp. 170–181, 2017.

[17] Andrew Y Ng, Stuart Russell, et al. Algorithms for inverse reinforcement learning. In *Proceedings of the 17th International Conference on Machine Learning*, No. 2 in ICML'00, pp. 663–670, 2000.

[18] Shade Ruangwan, Patanamon Thongtanunam, Akinori Ihara, and Kenichi Matsumoto. The impact of human factors on the participation decision of reviewers in modern code review. *Empirical Software Engineering*, Vol. 24, No. 2, pp. 973–1016, 2019.

[19] Patanamon Thongtanunam, Shane McIntosh, Ahmed E Hassan, and Hajimu Iida. Review participation in modern code review: An empirical study of the android, qt, and openstack projects. *Empirical Software Engineering*, Vol. 22, No. 2, pp. 768–817, 2017.

[20] Thomas Bock, Angelika Schmid, and Sven Apel. Measuring and modeling group dynamics in open-source software development: A tensor decomposition approach. *ACM Transactions on Software Engineering and Methodology*, Vol. 31, No. 2, pp. 1–50, 2021.

[21] Peter C Rigby, Seth Rogers, Sadruddin Saleem, Parth Suresh, Daniel Suskin, Patrick Riggs, Chandra Maddila, Nachiappan Nagappan, and Audris Mockus. Improving code reviewer recommendation: Accuracy, latency, workload, and bystanders. *ACM Transactions on Software Engineering and Methodology*, Vol. 35, No. 1, pp. 1–23, 2025.

[22] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, ICCV'17, pp. 2980–2988, 2017.